

Internet Security Protocols & Standards

Introduction to Web Applications

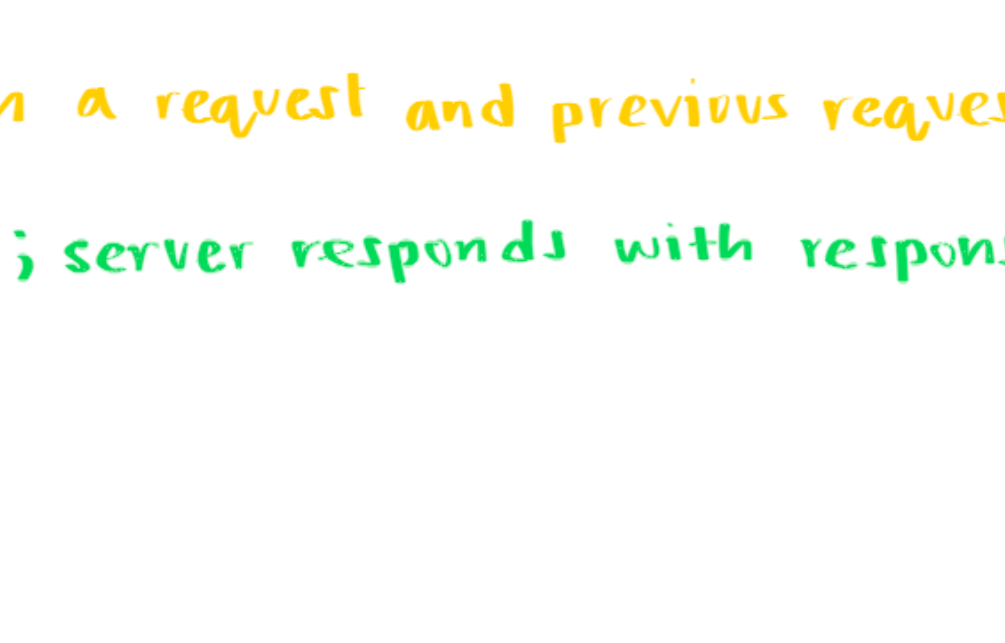
What are Web Applications?

- ↳ a computer program that utilizes web browsers & web technology to perform tasks over the internet
- ↳ how does a web app work?

1. User triggers a request to the **web server** over the **Internet**, either through a web browser or the application's user interface
2. **Web server** forwards this request to the appropriate **web application server**
3. **Web application server** performs the requested task – such as querying the **database** or processing the data – then generates the results of the requested data
4. **Web application server** sends results to the **web server** with the requested information or processed data
5. **Web server** responds back to the client with the requested information that then appears on the user's display

Web Browsing (HTTP)

- ↳ Application: Web Browsing
- ↳ Software: web browser (Firefox, Chrome, E. Safari)
- ↳ Protocol: HTTP
- ↳ client: web browser
- ↳ server: web server (Apache, IIS, nginx, lighttpd)
- ↳ HTTP is a request/response protocol
- ↳ stateless; no dependence between a request and previous request
- ↳ User Agent (client) sends request; server responds with response
- ↳ default port: 80



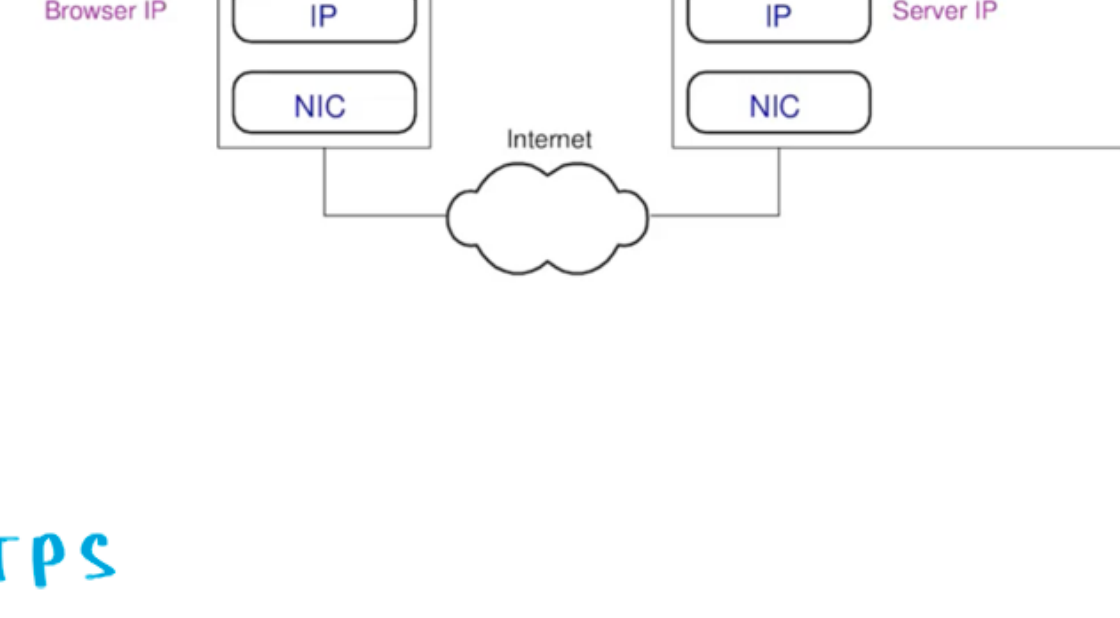
HTTP Messages

- ↳ request messages
- ↳ Start line: Method URL Version
- ↳ Methods:
 - GET: retrieve the resource at the specific URL
 - HEAD: same as GET, except do not return message body (only header)
 - OPTIONS: retrieve options available for resource or server
 - POST: asks server to accept and process the attached data at the resource
 - ...
- ↳ Version: version of HTTP, e.g. HTTP/1.0, HTTP/1.1
- ↳ response messages
- ↳ Start line: Version StatusCode StatusReason
- ↳ Status Codes and Reasons:
 - 100: Continue (the client should continue with its request)
 - 200: OK (the request succeeded)
 - 301: Moved Permanently (the requested resource has a new URL)
 - 304: Not Modified (resource hasn't changed since last request, client should use cached copy)
 - 401: Unauthorized (request must include user authentication)
 - 403: Forbidden (request was understood, but server refuses to process it)
 - 404: Not Found (server cannot find resource at requested URL)
 - 503: Service Unavailable (server currently unable to handle request, e.g. server is too busy)

- ↳ Date
- ↳ Authorization
- ↳ Host
- ↳ User Agent
- ↳ Accept-Charset, Accept-Encoding, Accept-Language

HTTP Applications

- ↳ can be static or dynamic
- ↳ many applications use dynamic content



HTTP vs HTTPS

How does HTTP work?

- ↳ browser — malicious user — server
- ↳ uses Wireshark or tcpdump
- ↳ everything sent in cleartext (GET, POST); can be read

HTTP Security Issues

- ↳ No data confidentiality
- ↳ no mutual authentication
- ↳ No data user confidentiality

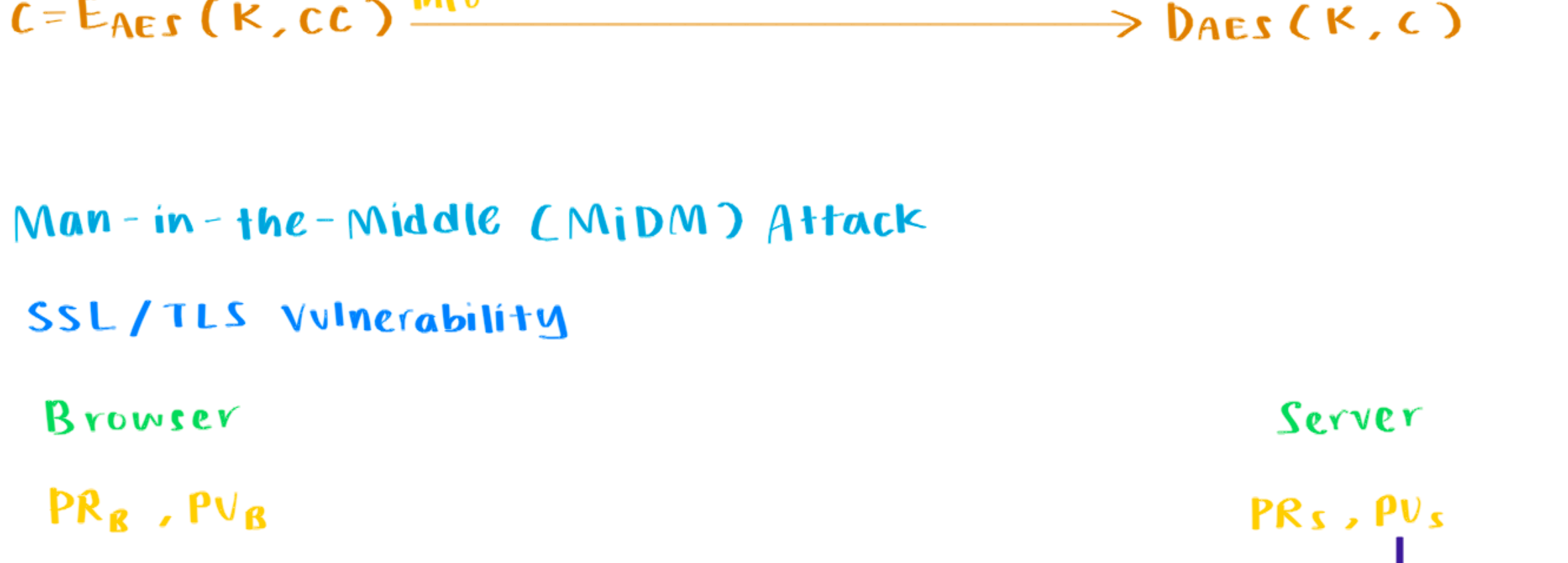
Using HTTPS

- ↳ encryption with HTTPS
- ↳ browser sure it is communicating with intended server
 - ↳ uses digital certificates
- ↳ Server sure it is communicating with intended user (HTTP also has this feature)
 - ↳ password authentication, session management
- ↳ action of user (browser) are kept private
 - ↳ anonymity services
- ↳ HTTPS: HTTP over SSL (or TLS)
 - ↳ web server listens on port 443
 - ↳ server authenticated using certificate (using SSL)
 - ↳ client authenticated using password (using HTTP) } mutual auth

Introducing SSL/TLS

- ↳ Secure Sockets Layer originated in Netscape Web browser
- ↳ Transport Layer Security standardised by IETF
- ↳ SSL v3 & TLS almost the same

How does SSL/TLS work?



Man-in-the-Middle (MiDM) Attack

SSL/TLS Vulnerability

- ↳ Browser: PR_B, PU_B
- ↳ Server: PR_S, PU_S
- ↳ PU_S
- ↳ K_BS
- ↳ K_BS

MiTM Attack

- ↳ Browser: PR_B, PU_B
- ↳ Attacker: PR_A, PU_A
- ↳ Server: PR_S, PU_S
- ↳ PU_A
- ↳ K_BS = E(PU_A, K_AB) → K_AB
- ↳ K_AB

Countermeasure (Certificates)

- ↳ use digital certificates to ensure it is Server we are talking to

Digital Certificates

Web Browsing Authentication & Encryption

- ↳ use public key cryptography to exchange secret key
 - ↳ RSA/DSA
 - ↳ Diffie-Hellman
 - ↳ Elliptic curve
- ↳ once secret key is exchanged, use symmetric key encryption
 - ↳ AES, RC4, 3DES...
- ↳ ex: if server sends browser its RSA public key, how can browser be sure this is the server's key?
 - ↳ get trusted third party to confirm

Digital Certificates

- ↳ Step 1: Server Obtains Digital Certificate
 - Server (owner) creates key pair: (PU_S, PR_S)
 - Server confirms identity, ID_S, with trusted third party called Certificate Authority
 - CA issues server with a digital certificate by signing relevant info:
$$C_S = ID_S || PU_S || T, E(PR_{CA}, H(ID_S || PU_S || T))$$
 - A timestamp, T, can be used to determine how long the certificate is valid
 - X.509 specifies standard format of certificates
- ↳ Step 2: Server Sends Digital Certificate to Browser
 - When server initiates communications with server, server responds with C_S
 - Browser verifies signature using PU_CA
 - Assumes browser already knows and trusts PU_CA
 - PU_CA is stored in self-signed certificate:
$$C_{CA} = ID_{CA} || PU_{CA} || T, E(PR_{CA}, H(ID_{CA} || PU_{CA} || T))$$
 - Once verified, browser can choose secret value and send it encrypted using PU_S to server

Certificates in Practice

How does a server obtain a certificate?

- ↳ Prove identity to CA by:
 - Domain validation
 - Extended validation
- ↳ Free and commercial services

How does browser obtain CA certificate?

- ↳ Pre-loaded into browsers
- ↳ Hierarchy of certificates is supported

What if CA certificate is not in browser?

- ↳ Browsers commonly present warning to user

Security Issues with Digital Certificates

- ↳ Identity verification of server (owners)
- ↳ Security of CA private key
- ↳ Pre-loaded certificates by browser publisher
- ↳ Response when invalid certificate received
- ↳ Algorithms used in certificates should be strong

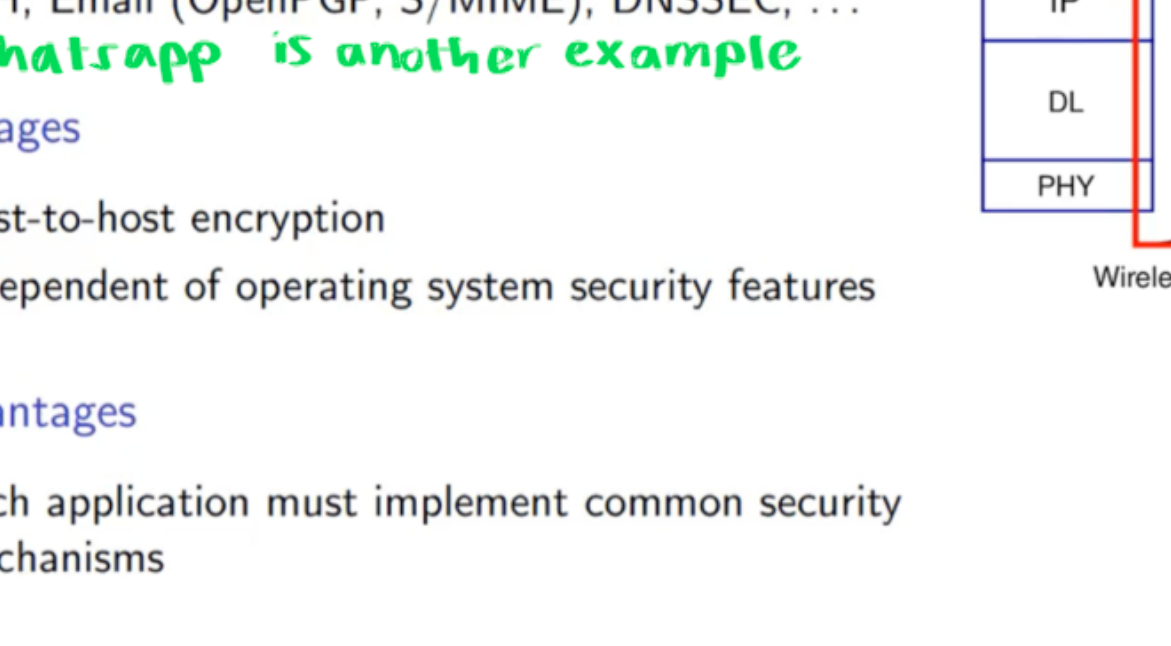
X.509 Certificates

- ↳ X.509 certificate format includes:
 - Version of X.509 certificate
 - Serial number unique to the issuer (CA)
 - Signature algorithm
 - Issuer's name and unique identifier
 - Period of validity (start time, end time)
 - Subject's name and unique identifier
 - Subject's public key information: algorithm, parameters, key
 - Signature
- ↳ Certificates may be revoked before expiry
 - CA signs a Certificate Revocation List (CRL), which is publicly available

Internet Security

Introduction to Internet Security

- ↳ securing communications across the Internet can be performed at different layers
- ↳ each layer adds their own header



Application Level Security

Application (protocol) implements its own security mechanisms

Examples

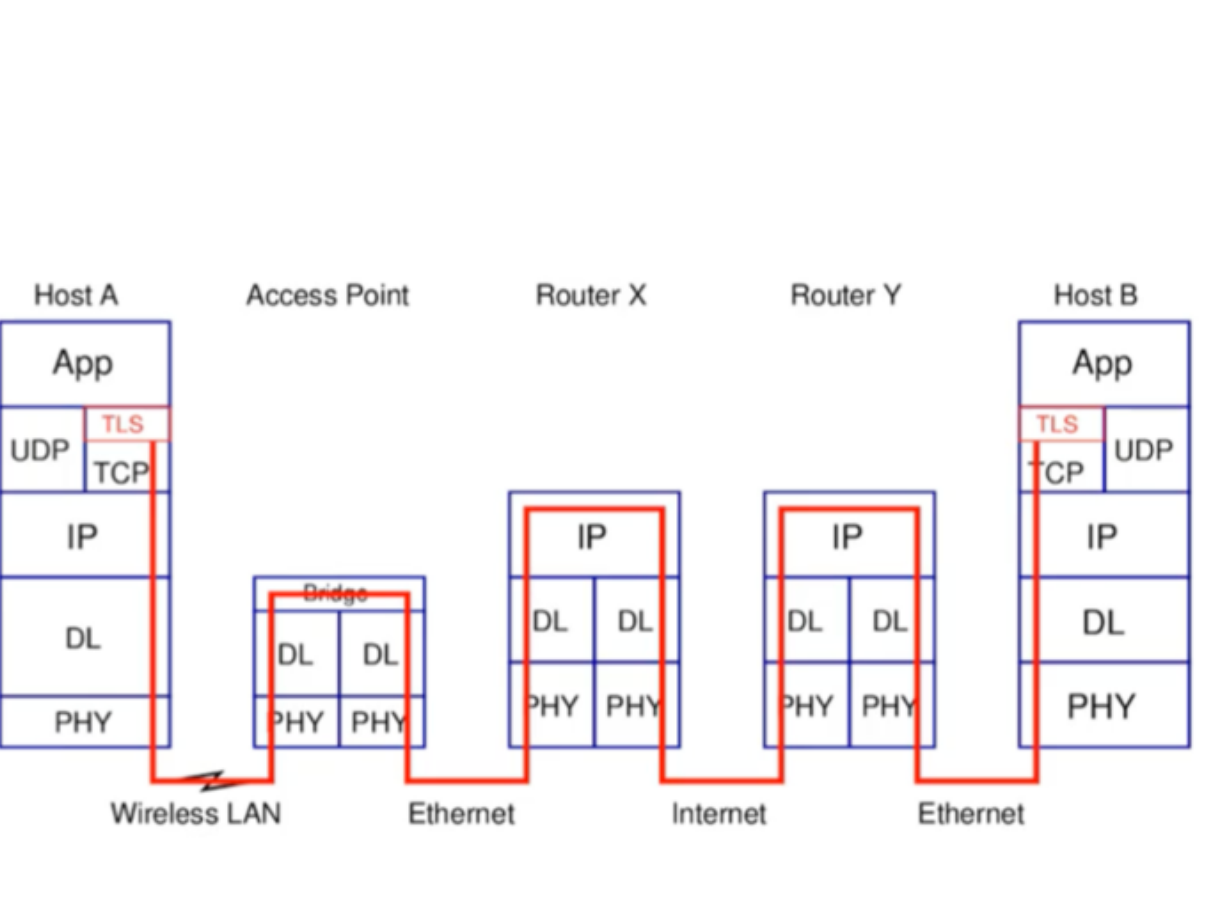
- ↳ SSH, Email (OpenPGP, S/MIME), DNSSEC, ...
- ↳ **whatsapp is another example**

Advantages

- ↳ Host-to-host encryption
- ↳ Independent of operating system security features

Disadvantages

- ↳ Each application must implement common security mechanisms



Transport Level Security

Application uses OS provided library for security

Examples

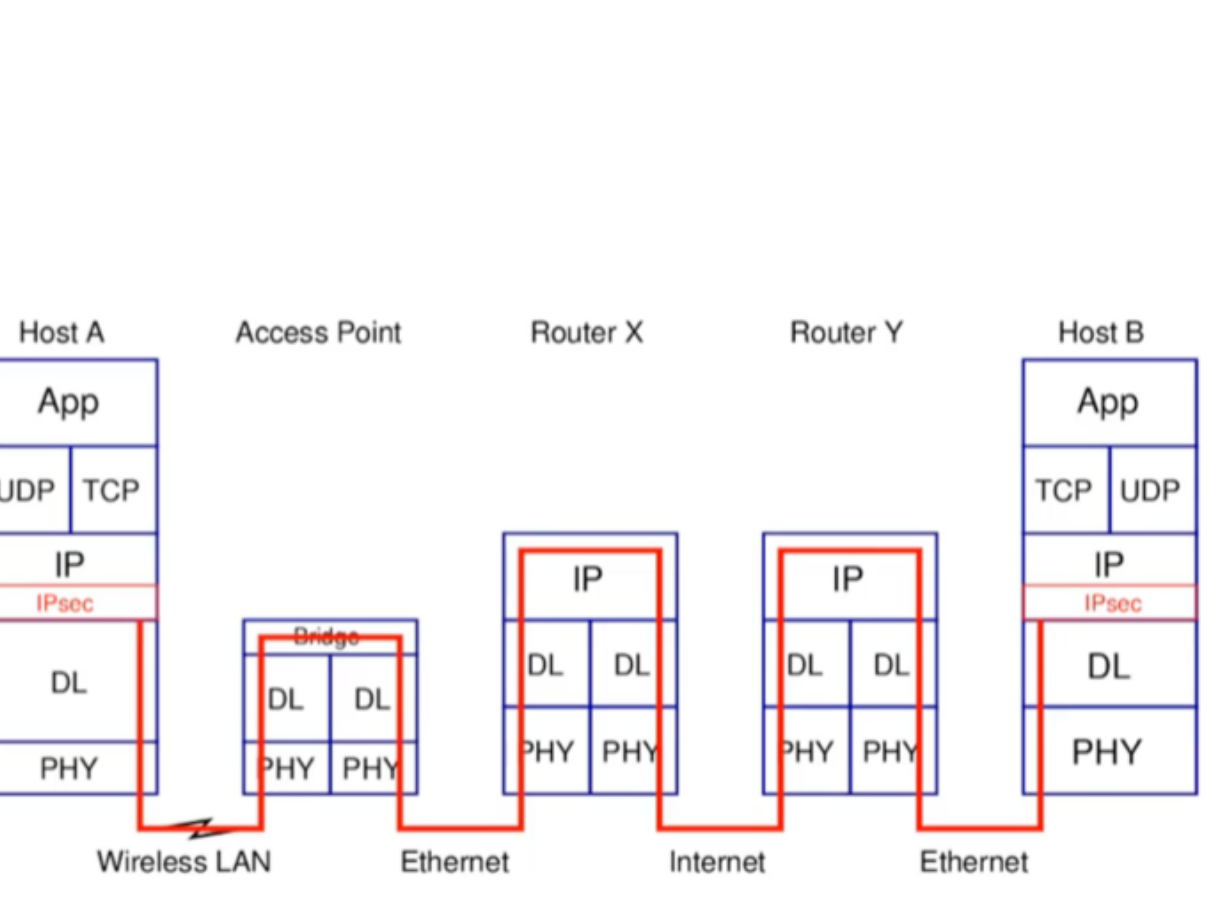
- ↳ TLS/SSL for TCP-based applications, e.g. HTTPS, IMAPS, FTPS, SMTPS
- ↳ DTLS, SRTP for other transport protocols

Advantages

- ↳ Host-to-host encryption
- ↳ Simpler applications; no need to implement complex security mechanisms

Disadvantages

- ↳ Only applies for specific transport protocols
- ↳ Applications must be implemented to use OS API



Network Level Security

Computer configured to apply security mechanisms to IP packets

Examples

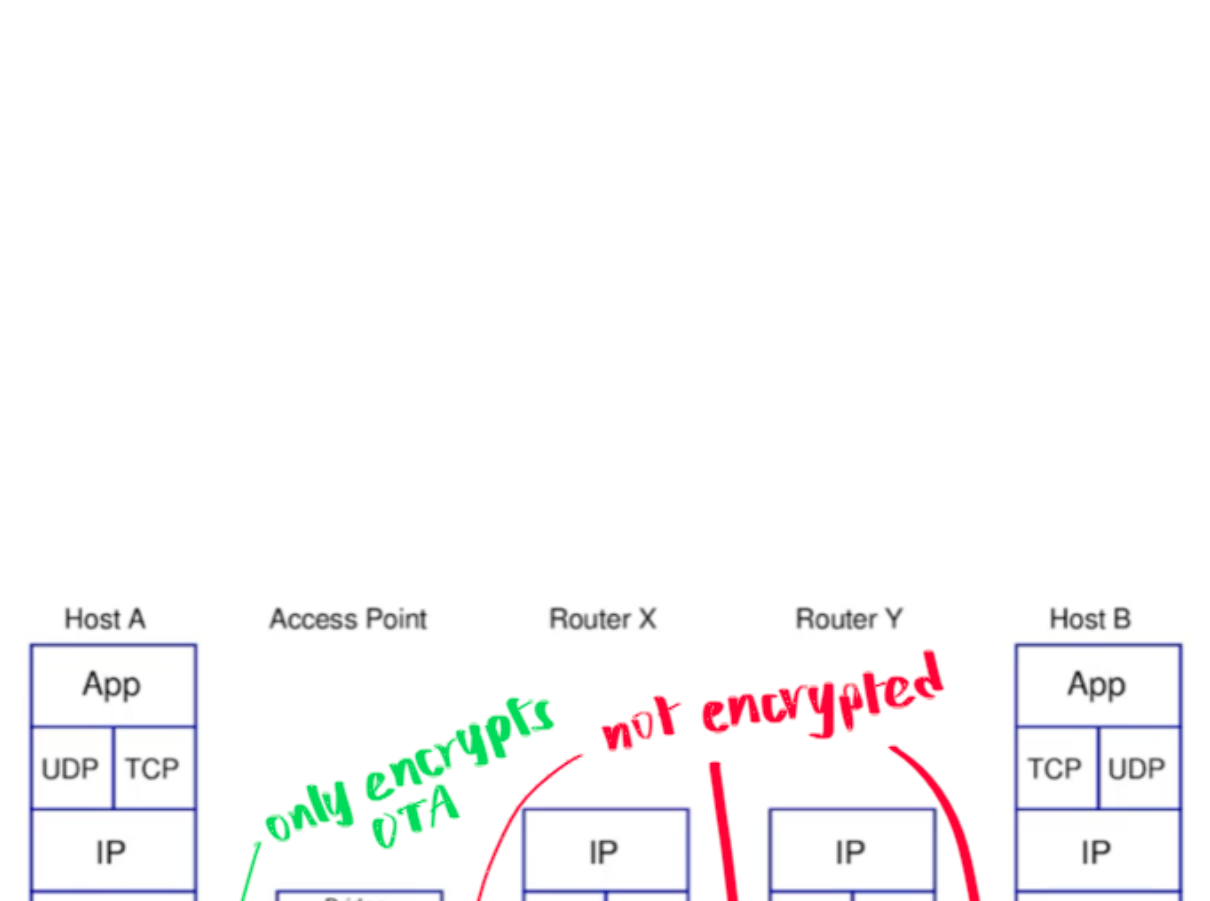
- ↳ IPSec

Advantages

- ↳ Supports all applications and transport protocols
- ↳ Can be host-to-host encryption

Disadvantages

- ↳ Requires support and configuration in OS



Link Level Security

Examples

- ↳ WEP/WPA in wireless LANs, Bluetooth, ZigBee encryption, GSM A3/A5/A8, ...

Advantages

- ↳ Applies to all data sent across link, independent of application, transport, network protocols

Disadvantages

- ↳ Encryption only across the link

